

Outmaneuver

Cattolico Giuseppe, Muller Arthur, Pulzoni Alessio, Spinaci Gianmarco

25 giugno 2026

Indice

1	Analisi	3
1.1	Descrizione e requisiti	3
1.2	Modello del Dominio	4
2	Design	7
2.1	Architettura	7
2.2	Design dettagliato	9
2.2.1	Cattolico Giuseppe	9
2.2.2	Muller Arthur	14
2.2.3	Pulzoni Alessio	14
2.2.4	Spinaci Gianmarco	16
3	Sviluppo	20
3.1	Testing automatizzato	20
3.2	Note di sviluppo	22
3.2.1	Cattolico Giuseppe	22
3.2.2	Muller Arthur	22
3.2.3	Pulzoni Alessio	22
3.2.4	Spinaci Gianmarco	22
4	Commenti finali	23
4.1	Autovalutazione e lavori futuri	23
4.1.1	Cattolico Giuseppe	23
4.1.2	Muller Arthur	24
4.1.3	Pulzoni Alessio	24
4.1.4	Spinaci Gianmarco	24
A	Guida utente	26
A.1	Avvio e menù principale	26
A.2	Comandi	27
A.3	Svolgimento della partita	28

A.4	Negozio	29
A.5	Classifica e fine della partita	30
B	Esercitazioni di laboratorio	32
B.1	giuseppe.cattolico@studio.unibo.it	32
B.2	arthuristvan.muller@studio.unibo.it	32
B.3	alessio.pulzoni@studio.unibo.it	32
B.4	gianmarco.spinaci3@studio.unibo.it	32

Capitolo 1

Analisi

1.1 Descrizione e requisiti

OutManeuver è un videogioco arcade 2D, sviluppato in Java, ispirato al titolo mobile Missiles. Il giocatore pilota un velivolo in uno spazio aperto con l'obiettivo di sopravvivere il più a lungo possibile, schivando una quantità crescente di missili a ricerca automatica e inducendoli a collidere tra loro. Per missile a ricerca automatica si intende un proiettile dotato di un algoritmo di inseguimento autonomo (steering behavior) che aggiorna continuamente la propria traiettoria in direzione del velivolo del giocatore. Durante la partita compaiono a schermo entità collezionabili di tipologie distinte, che il giocatore può raccogliere per ottenere vantaggi temporanei o per incrementare il proprio punteggio. Al di fuori della partita, una sezione gestionale — il Negozio — consente di acquistare velivoli potenziati spendendo la valuta di gioco accumulata. La difficoltà aumenta progressivamente nel tempo, sia nella frequenza di generazione dei missili sia nella loro pericolosità media. La partita termina nel momento in cui un missile colpisce il velivolo in assenza di uno scudo attivo; il punteggio conseguito viene infine confrontato con la classifica dei record.

Requisiti funzionali

- Il sistema genera missili ai bordi dell'area di gioco con frequenza crescente nel tempo, ciascuno dei quali insegue autonomamente il velivolo del giocatore.
- Il sistema rileva i contatti tra le entità di gioco: missile-velivolo, missile-missile e velivolo-collezionabile.

- Il sistema prevede più tipologie di missili, ciascuna caratterizzata da un comportamento distinto.
- Il sistema genera periodicamente entità collezionabili che il giocatore può raccogliere per ottenere effetti di gioco.
- Il sistema presenta un menù principale con le opzioni di avvio partita e di uscita dall'applicazione.
- Il sistema fornisce un HUD (Heads-Up Display, ovvero un insieme di informazioni sovrapposte all'area di gioco e visibili durante la partita) che mostra il punteggio aggiornato in tempo reale, e una schermata di Game Over al termine della partita.
- Il sistema permette di acquistare velivoli potenziati, dalle caratteristiche superiori, spendendo la valuta accumulata nelle partite.
- Il sistema salva su file e visualizza una classifica dei punteggi massimi conseguiti.

Requisiti non funzionali

- Il sistema garantisce una risposta fluida e reattiva agli input del giocatore in ogni condizione di gioco.
- Il sistema è eseguibile sui principali sistemi operativi desktop (Windows, macOS, Linux).
- Il sistema rende immediatamente comprensibili le proprie meccaniche, risultando intuitivo per qualsiasi giocatore.
- Il sistema adatta dinamicamente la propria interfaccia e l'area di gioco alle dimensioni della finestra.

1.2 Modello del Dominio

Il dominio di OutManeuver è costituito da un insieme di entità che interagiscono all'interno di un'Area di Gioco. L'entità centrale è il Velivolo, controllato dal giocatore, che si muove continuamente nell'area virando a destra o a sinistra. Il Velivolo può trovarsi in due stati: vulnerabile oppure protetto, quando è attivo uno Scudo. Ne esistono varianti potenziate, acquistabili nel Negozio, che si differenziano per velocità di punta, raggio di virata e dimensione della propria area di collisione. L'area è popolata da Missili,

entità autonome che inseguono il Velivolo aggiornando di continuo la propria direzione di movimento. I Missili si distinguono in tipologie diverse, ciascuna caratterizzata da velocità, manovrabilità e comportamento propri: alcuni inseguono il bersaglio, altri procedono in linea retta, altri rimbalzano sui bordi dell'area, altri ancora producono effetti particolari sugli altri missili. Ogni Missile può collidere con il Velivolo, determinando la condizione di Game Over in assenza di uno Scudo attivo, oppure con un altro Missile, provocando la distruzione reciproca di entrambi. A intervalli di tempo compaiono nell'area le Entità Collezionabili, che il Velivolo raccoglie transitandovi sopra. Si distinguono in tre tipologie: il Boost di Velocità, che aumenta temporaneamente la velocità del Velivolo; la Stella, che incrementa il punteggio del giocatore; lo Scudo, che conferisce al Velivolo una protezione a tempo. Lo Scudo non si consuma al singolo impatto: finché è attivo, qualunque contatto con un missile non provoca il Game Over, indipendentemente dal numero di impatti subiti; la protezione decade solo allo scadere della propria durata. Lo stato della partita è tenuto dalla Sessione di Gioco, che conserva il punteggio corrente e il tempo di sopravvivenza e governa le transizioni tra gli stati del gioco (menu, partita in corso, pausa, game over). Al termine della partita il punteggio viene confrontato con la Classifica, che conserva su file locale i migliori risultati delle sessioni precedenti. Esiste infine una sezione gestionale, il Negozio (o Hangar), accessibile al di fuori della Sessione di Gioco. Al suo interno il giocatore spende la valuta accumulata durante le partite per sbloccare e acquistare Velivoli potenziati: a differenza del velivolo di base, queste varianti presentano caratteristiche tecniche superiori (ad esempio una maggiore velocità di punta o un raggio di virata più stretto). Ogni velivolo acquistato entra a far parte della dotazione del giocatore e può essere equipaggiato prima di una nuova partita. Un aspetto particolarmente critico del dominio riguarda la relazione tra il Missile e il Velivolo: il comportamento di inseguimento deve bilanciare la manovrabilità dei missili in modo che la sfida risulti equa e progressiva. Questo equilibrio è intrinseco alla natura del problema e non dipende da scelte implementative, ma dalla definizione stessa del dominio.

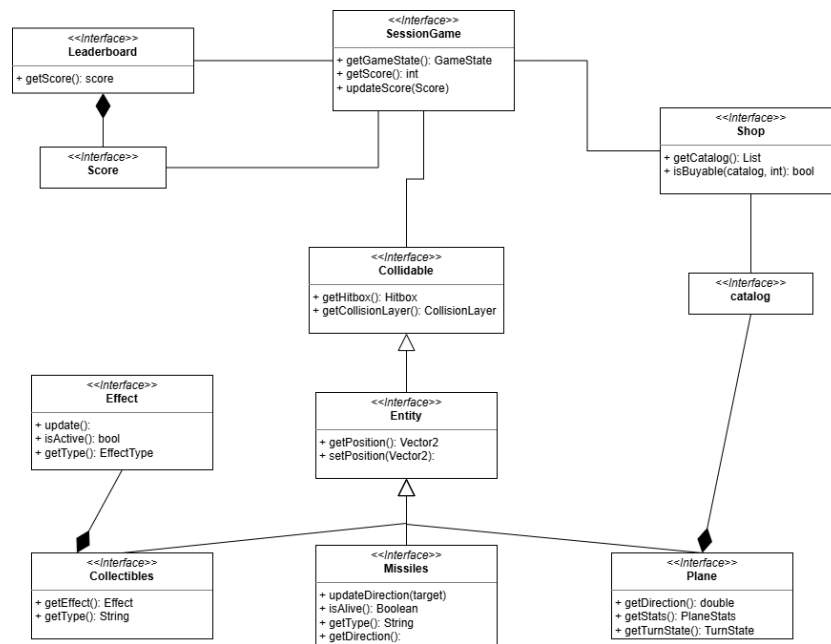


Figura 1.1: Schema UML dell'analisi del dominio, con rappresentate le entità principali ed i rapporti fra loro

Capitolo 2

Design

2.1 Architettura

OutManeuver adotta il pattern architetturale MVC, strutturato per supportare un'applicazione in tempo reale tramite una netta separazione delle responsabilità e dipendenze strettamente unidirezionali.

I Tre Livelli Architetturali

- **Model:** Gestisce lo stato della partita, le regole fisiche e i sistemi secondari (come negozio e classifiche). Non è un blocco monolitico, ma espone entry point distinti in base alle responsabilità. Delega il caricamento dei parametri di configurazione a un pattern Repository, mantenendo il dominio puro e isolato dalle logiche di salvataggio. Non ha alcuna conoscenza dei livelli superiori.
- **Controller** (Orchestrazione e Game Loop): È il motore del sistema. Gestisce il game loop continuo, l'aggiornamento fisico delle entità, il rilevamento delle collisioni e la difficoltà (spawning). Traduce gli input dell'utente in azioni sul Model e funge da unico intermediario per l'estrazione dei dati.
- **View** : È una componente puramente passiva. Si limita a renderizzare a schermo i dati pronti che riceve e a raccogliere gli input dell'utente, occupandosi anche della navigazione tra le varie schermate (menu, gioco, game over). Non interroga mai il Model direttamente.

La comunicazione tra i livelli è bidirezionale ma sempre mediata dal Controller, garantendo un forte disaccoppiamento:

Dalla View al Controller (Eventi): L'interfaccia notifica le azioni dell'utente sotto forma di eventi astratti. Il Controller interpreta questi eventi e applica le conseguenti modifiche al Model.

Dal Controller alla View (Snapshot Immutabili): A ogni ciclo del game loop, il Controller "fotografa" lo stato del Model e costruisce un pacchetto di dati di sola lettura (DTO/Snapshot) che invia alla View per il rendering.

L'adozione di questi snapshot immutabili è il cuore dell'architettura in tempo reale di OutManeuver: permette al motore logico (su un thread) e all'interfaccia grafica (su un altro thread) di lavorare in parallelo. Poiché le due parti si scambiano solo dati in sola lettura, si garantisce un ambiente thread-safe privo di conflitti, consentendo inoltre di sostituire l'intero motore grafico senza dover toccare minimamente la logica di gioco.

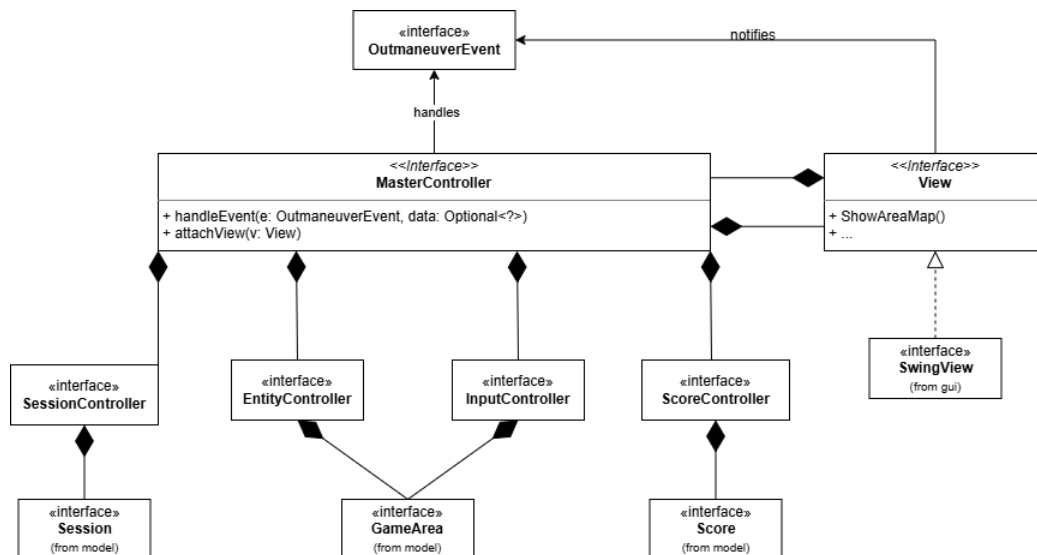


Figura 2.1: Schema UML del diagramma architetturale, in cui si evidenzia la separazione tra le parti di Model, View, Controller e come i controller si interfacciano col Model

2.2 Design dettagliato

2.2.1 Cattolico Giuseppe

Creazione e orchestrazione della User Interface (UI)

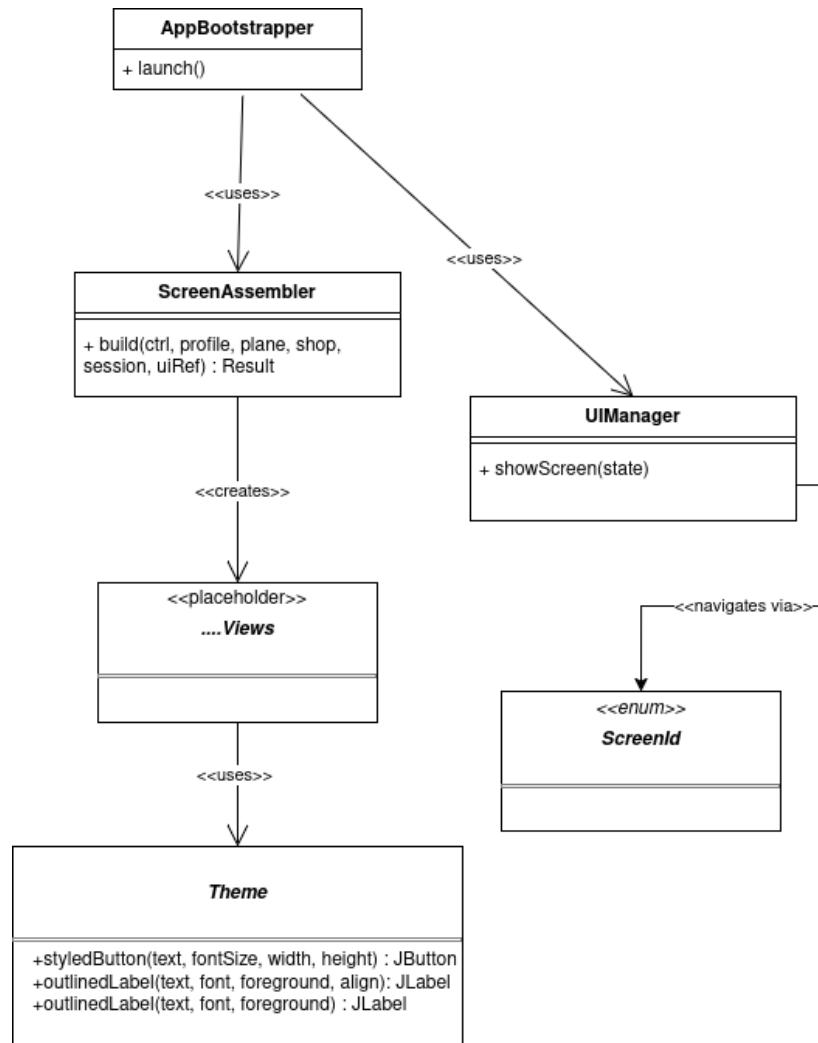


Figura 2.2: Schema UML dell'orchestrazione della User Interface

Problema L'applicazione è composta da molteplici schermate (MainMenu-View, ShopView, GameOverView, ecc.). Ciascuna di queste viste necessita di configurazioni specifiche, metriche di scaling (ScreenMetrics), stili grafici (definiti in Theme) e svariati listener per le interazioni. Inizializzare queste

istanze in modo sparso durante il ciclo di vita dell'applicazione o all'interno della singola classe di avvio avrebbe generato un forte accoppiamento, rendendo il codice rigido e violando il principio di singola responsabilità (SRP).

Soluzione Si è deciso di accentrare la responsabilità della creazione delle viste in un componente dedicato (ScreenAssembler), che pre-costruisce tutte le schermate fornendo loro le corrette dipendenze fin dall'avvio. Per la gestione della navigazione, anziché permettere alle singole viste di distruggersi e ricrearsi a vicenda, si è introdotto un UIManager. Questo componente mantiene il controllo del routing tramite un approccio a stati finiti descritto dall'enumerazione ScreenId. L'alternativa di un approccio lazy loading (creare la vista solo quando richiesta) è stata scartata a favore della costruzione up-front dell'Assembler, in quanto garantisce fin dall'avvio che tutte le metriche e le dipendenze (come il profilo e il negozio) siano valide, migliorando la fluidità nel passaggio tra le schermate.

Design Pattern In questa sezione è stato reificato il pattern creazionale Factory (in una sua accezione di Assembler per Dependency Injection). La classe ScreenAssembler agisce da Factory: il suo metodo build(...) nasconde la complessa logica di istanziazione delle varie sottoclassi concrete della GUI, restituendo un Result incapsulato. In aggiunta, la separazione visiva e logica degli elementi grafici comuni sfrutta la classe Theme tramite costanti statiche e metodi di utilità, accentrando la definizione dello stile per agevolare future reskin dell'applicativo (modifica di un solo file per alterare l'intera estetica).

Gestione economica: Negozio e Profilo Giocatore

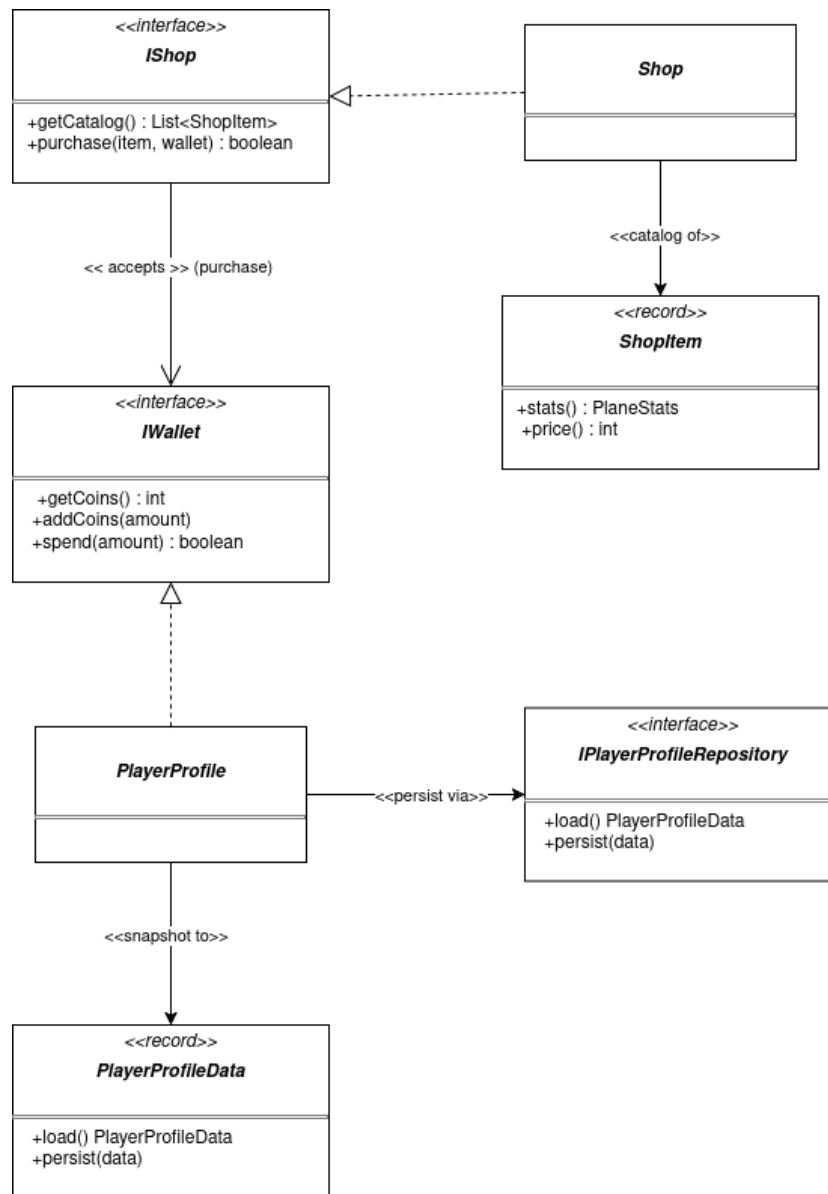


Figura 2.3: Schema UML della gestione economica e del salvataggio dei dati utente

Problema Il negozio virtuale (Shop) ha il compito di gestire il catalogo (ShopItem) e processare gli acquisti da parte del giocatore. Affinché un acquisto vada a buon fine, il negozio deve poter leggere e scalare le monete dell'utente. Tuttavia, passare direttamente l'istanza concreta di PlayerProfile

allo Shop avrebbe esposto quest'ultimo a dettagli di dominio non pertinenti (come lo storico dei punteggi, le logiche di salvataggio file, ecc.), creando un legame rigido e riducendo la riusabilità del componente negozio.

Soluzione Si è applicato il principio di Inversione delle Dipendenze (DIP) e della Segregazione delle Interfacce (ISP). È stata creata un'interfaccia ristretta `IWallet`, che espone solo i metodi finanziari (`getCoins`, `addCoins`, `spend`). La classe `PlayerProfile` implementa questa interfaccia. Quando il metodo `purchase` dello Shop viene invocato, richiede esclusivamente un parametro di tipo `IWallet`. Parallelamente, per slegare del tutto il profilo utente dal file system, l'operazione di salvataggio è stata delegata all'astrazione `IPlayerProfileRepository`. In questo modo, è possibile testare le logiche del negozio o del profilo fornendo implementazioni fittizie (mock) senza dover gestire veri file JSON.

Design Pattern Viene implementato il Repository Pattern tramite l'interfaccia `IPlayerProfileRepository` (e la sua reificazione concreta `JsonPlayerProfileRepository` nel codice sorgente). Questo astrae totalmente i dettagli di accesso ai dati. Inoltre, tramite l'estrazione di `IWallet`, si realizza strutturalmente il Dependency Injection a livello di metodo: lo Shop inietta le logiche di spesa su un'entità astratta, dimostrando un disaccoppiamento ideale orientato alle interfacce.

Rendering State e Session Snapshot

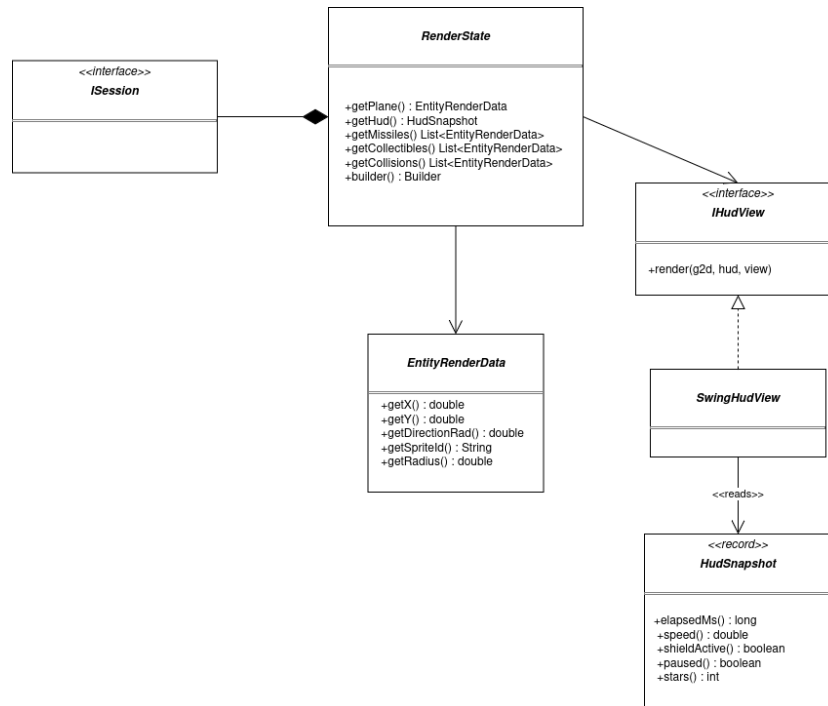


Figura 2.4: Schema UML relativo al trasferimento dello stato al motore di rendering grafico

Problema Il motore grafico (la vista del gioco in tempo reale) deve disegnare gli elementi sullo schermo ad ogni tick temporale. Passare direttamente al renderer le entità mutabili del modello di gioco (Session) avrebbe violato la barriera architetturale tra Model e View, permettendo (anche accidentalmente) alla grafica di modificare lo stato del gioco. Inoltre, la costruzione dello stato da inviare all'HUD e al canvas di gioco prevede numerosi parametri (posizione aereo, liste di missili, liste di collisioni e dati di punteggio), che avrebbero reso eventuali costruttori illeggibili e soggetti ad errori (anti-pattern Telescoping Constructor).

Soluzione Si è adottato un approccio basato sull'immutabilità e sulla costruzione a passi. Si estrae lo stato logico in un'entità in sola lettura chiamata HudSnapshot (per l'UI overlay) e in un RenderState per le posizioni spaziali. Per ovviare alla complessità di istanziazione di RenderState (che richiede svariate liste), si è integrato un costruttore fluido. Questa soluzione garantisce che la Vista riceva esclusivamente i dati strettamente necessari per il

rendering in un formato non alterabile, mantenendo integro il pattern MVC di base.

Design Pattern È stato utilizzato diffusamente il pattern creazionale Builder. Il Builder è reificato dalla classe nested statica `RenderStateBuilder` contenuta all'interno di `RenderState`. Il metodo costruttore della classe principale è privato, obbligando l'uso della sintassi fluida. Questo isola la logica di assemblaggio dal suo stato finale. Parallelamente, il record `HudSnapshot` agisce come un pattern architetturale DTO (Data Transfer Object), utilizzato puramente per trasferire informazioni sicure, isolate e immutabili tra il Model (che governa la sessione temporale e il punteggio) e lo strato puramente visivo di `SwingHudView`.

2.2.2 Muller Arthur

Infrastruttura per la configurazione dei dati di gioco tramite JSON

2.2.3 Pulzoni Alessio

Gerarchia dei missili e comportamenti differenti

Problema I missili condividono un ciclo di vita comune (inseguimento del bersaglio, movimento, gestione del tempo di vita e del rallentamento), ma ogni tipo si comporta diversamente: chi insegue, chi procede in linea retta, chi rimbalza sui bordi, chi rallenta gli altri, chi regge due colpi.

Soluzione Ho applicato il Template Method Pattern. La classe astratta `MissileImpl` definisce lo scheletro invariante dell'aggiornamento nel metodo `update()`, scomposto in `shouldSkipUpdate()` → `steer()` → `move()`. I passi che non variano (`move`, `shouldSkipUpdate`) sono dichiarati `final`, mentre il punto di variazione è il metodo `steer()` (hook), che ogni tipo ridefinisce: `StandardMissile` insegue il bersaglio con virata limitata, `SniperMissile` e `BounceMissile` non sterzano, e così via. Ho preferito il Template Method al duplicare la logica in ogni classe (rispetto del principio DRY), e perché l'aggiunta di un nuovo tipo non richiede di modificare quelli esistenti (OCP). Il movimento è scalato per il `dt` (delta time), rendendo la simulazione indipendente dal frame-rate.

Catalogo dei tipi e loro creazione

Problema Occorre creare il missile concreto a partire dal suo tipo, tenendo al contempo in un unico luogo i metadati di gioco di ciascun tipo (peso di difficoltà, tempo di sblocco, numero massimo a schermo).

Soluzione Ho realizzato l'enum `MissileKind` come catalogo: ogni costante porta con sé i propri metadati e un riferimento al costruttore del tipo, tipizzato come `BiFunction<Vector2, MissileData, Missile>` (ad esempio `StandardMissile::new`). Il metodo `create()` si limita ad applicare la `BiFunction`, realizzando di fatto una `Factory` priva di switch o catene di if. Aggiungere un nuovo tipo di missile significa aggiungere una sola riga all'enum (OCP), e tutti i metadati di un tipo restano coesi in un unico punto. Ho preferito questa soluzione a una factory basata su switch perché elimina il punto di modifica centralizzato e sfrutta i method reference di Java.

Difficoltà progressiva: scelta adattiva del tipo da generare

Problema Il requisito impone che la difficoltà cresca nel tempo in modo equo e graduale: il sistema deve decidere quale tipo di missile generare, e non sceglierlo in modo puramente casuale.

Soluzione Ho progettato `MissileSpawnDirector` come classe pura: il suo unico stato è il generatore `Random`, il che la rende testabile in isolamento con un seme fissato. La scelta è la composizione di tre funzioni: una sigmoide definisce la difficoltà media desiderata al tempo t ; una tangente iperbolica misura il divario, normalizzato in $[-1, 1]$, tra la difficoltà attuale e quella desiderata; una funzione softmax (di Boltzmann) converte tale divario e la "complessità" di ciascun tipo in pesi di probabilità, dai quali estraggo poi con una roulette pesata. Il meccanismo è auto-bilanciante (se il campo è troppo difficile spinge verso i tipi più facili, e viceversa) e applica filtri di sblocco temporale, di numero massimo a schermo e di numero minimo di missili attivi. Costituisce il fulcro della difficoltà progressiva richiesta dai requisiti.

2.2.4 Spinaci Gianmarco

Gerarchia dei controller

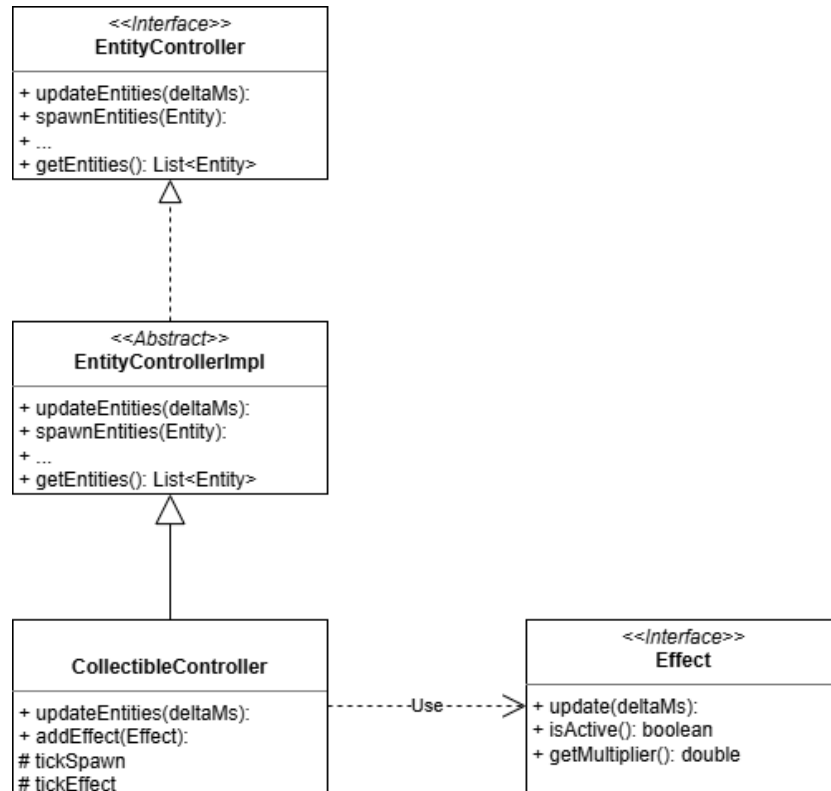


Figura 2.5: Rappresentazione Uml del Pattern Template Method per i controller delle entità e collectibles

Problema Il sistema richiede diversi tipi di controller per entità con comportamenti molto diversi: aerei, missili, collectible. Ogni controller condivide le operazioni di registrazione e rimozione delle entità dal motore di collisione, ma la logica di aggiornamento (updateEntities) differisce radicalmente da un tipo all'altro. Si vuole evitare la duplicazione di codice pur lasciando piena libertà a ciascun controller di definire la propria logica. Si vuole, inoltre, evitare di applicare gli effetti dei Collectibles direttamente sul Plane.

Soluzione È stato applicato il pattern Template Method. EntityControllerImpl è la classe astratta che definisce e implementa i metodi comuni ai sub-controller che gestiscono il che permette di gestire una lista unica di entità, lasciando updateEntities come metodo da sovrascrivere. CollectibleControllerImpl estende questa base e ridefinisce updateEntities orchestrando due sotto-tick: il primo si occupa di generare i Collectibles ai margini a intervalli regolari e il secondo che si occupa di tenere aggiornati gli effetti. CollectibleControllerImpl, mantiene una lista interna di Effect attivi che vengono aggiornati a ogni tick e rimossi alla scadenza, notificando sempre il listener tramite onInternalEvent in modo tale da non dover più applicare gli effetti direttamente sul Plane.

Motore di collisione e notifica eventi

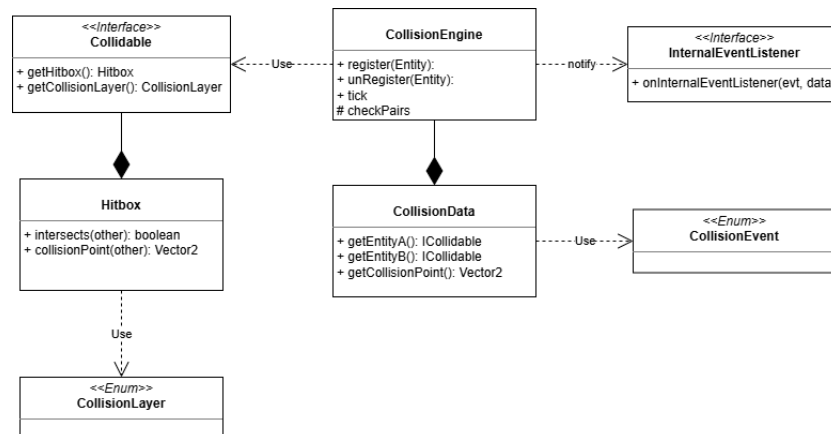


Figura 2.6: Rappresentazione Uml del pattern Observer per gli eventi di collisione

Problema Il CollisionEngine deve rilevare collisioni tra entità di layer diversi (Plane, Missile, Collectible) e comunicare il risultato al resto del sistema — ad esempio per distruggere un missile, applicare un effetto o comunicare

il Game Over del giocatore — senza dipendere direttamente dai controller che reagiranno a quegli eventi.

Soluzione È stato applicato il pattern Observer. CollisionEngine è l'observable: al momento della costruzione riceve un InternalEventListener (interfaccia funzionale, implementabile anche con lambda) a cui notifica ogni collisione rilevata tramite onInternalEvent(CollisionEvent, CollisionData). Chi registra il listener (tipicamente l'Event controller o il Master controller) funge da observer e smista l'evento al sottosistema corretto. Questo disaccoppia completamente il rilevamento della collisione dalla sua gestione. Il CollisionEngine a ogni tick() filtra le entità registrate per CollisionLayer, poi confronta le coppie rilevanti (missile×missile, missile×aereo, aereo×collectible) usando Hitbox.intersects(). Quando due hitbox si sovrappongono, calcola il punto di collisione e lo incapsula in un CollisionData, passandolo al listener.

Rendering delle esplosioni e dei collectible

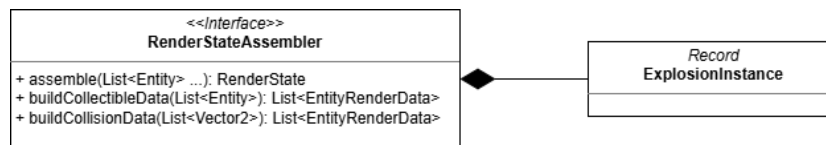


Figura 2.7:

Problema La view non deve mai accedere direttamente alle entità del modello: deve ricevere solo dati già pronti al rendering (posizione, sprite, raggio). Inoltre, le esplosioni non sono entità persistenti nel modello ma effetti visivi temporanei che devono vivere per un numero fisso di tick dopo una collisione, decadendo frame per frame. Come si assembla questo stato in modo pulito?

Soluzione RenderStateAssembler riceve in ingresso la lista delle entità vive, i parametri di HUD e la lista dei punti di collisione rilevati nel tick corrente, e produce un unico RenderState immutabile che la view si limita a consumare. Per le esplosioni, l'assembler mantiene internamente una lista di ExplosionInstance (record con posizione e contatore di tick). Ogni punto di collisione ricevuto genera una nuova istanza. Ad ogni chiamata ad assemble, le istanze vengono incrementate e quelle che superano il proprio tempo di vita vengono rimosse. La view riceve un EntityRenderData con sprite "explosion" e come campo direction il tick corrente, che può usare per animare il frame corretto dello sprite sheet. Per i collectible, l'assembler legge il tipo

dalla entità e costruisce il nome dello sprite come "collectible" + collectible-Type, delegando alla view solo il compito di caricare la risorsa dal nome. Le esplosioni non vengono mai passate alla view come entità "da rimuovere", ma semplicemente smettono di apparire nella lista quando superano il limite di tick. Eliminando per entrambi qualsiasi traduzione dal controller viene applicato il principio di Tell, Don't Ask: è il modello che sa qual è il proprio tipo, e l'assembler ne costruisce il nome direttamente, senza switch né instanceof.

Capitolo 3

Sviluppo

3.1 Testing automatizzato

Il modulo **Model** racchiude lo stato dell'applicazione, le entità di gioco e la logica di business core. I test ad esso associati garantiscono l'integrità dei dati, la correttezza dei calcoli vettoriali e la coerenza del ciclo di vita delle sessioni.

- **Vector2Test**: verifica le operazioni vettoriali (somma, scala, normalizzazione, prodotto scalare, angolo, costruzione da angolo) e l'immutabilità della classe.
- **GameSessionTest**: verifica le transizioni di stato valide e non valide e il corretto conteggio del tempo di sopravvivenza.
- **ScoreEntryTest**: verifica l'ordinamento decrescente delle voci della classifica.
- **ShopTest**: verifica il catalogo dei velivoli e la logica di acquistabilità.
- **PlaneImplTest**: verifica lo stato del velivolo (posizione, scudo, moltiplicatore di velocità).
- **JsonMissileRepositoryTest**: verifica il caricamento dei parametri dei missili da file JSON e la ricerca per tipo.
- **CollectiblesTest**: verifica l'effetto applicato da ciascun collezionabile.
- **PlayerProfileTest**: verifica la gestione del profilo (valuta, possesso dei velivoli).

Il modulo **Controller** funge da orchestratore del sistema, traducendo l'input dell'utente in azioni di gioco, gestendo le collisioni e coordinando l'evoluzione temporale dei vari sottosistemi.

- **MissileSpawnDirectorTest**: verifica le regole di selezione del tipo di missile — sblocco per tempo, vincolo di numero minimo di missili attivi (il clock) e robustezza dell'estrazione — usando un generatore con seme fisso per garantire la totale riproducibilità.
- **CollisionEngineTest**: verifica il rilevamento delle collisioni tra coppie di entità appartenenti ai diversi layer.
- **EntityControllerImplTest**: verifica la gestione delle entità (registrazione e rimozione, anche dal motore di collisioni).
- **CollectibleControllerImplTest**: verifica la generazione e la rimozione dei collezionabili.
- **HudControllerImplTest**: verifica la produzione dello snapshot dell'HUD (tempo, stelle, pausa).
- **InputControllerImplTest**: verifica la traduzione degli input della tastiera in direzioni astratte.
- **MasterControllerImplTest**: verifica l'orchestrazione del sistema e la gestione degli eventi.

Il modulo **View** gestisce la rappresentazione visiva e l'interfaccia grafica. I test verificano l'integrità del passaggio dei dati di rendering e il corretto comportamento dei componenti visuali e dei listener.

- **DtoTest**: verifica la correttezza e l'immutabilità dei DTO di rendering (`RenderState`, `EntityRenderData`).
- **SwingHudViewTest**, **ShopViewTest**, **GameOverViewTest**, **GameKeyListenerTest**: verificano alcune componenti dell'interfaccia (disegno dell'HUD, schermata del negozio, schermata di Game Over e gestione dei tasti).

3.2 Note di sviluppo

3.2.1 Cattolico Giuseppe

3.2.2 Muller Arthur

3.2.3 Pulzoni Alessio

3.2.4 Spinaci Gianmarco

Utilizzo della libreria.....

Capitolo 4

Commenti finali

4.1 Autovalutazione e lavori futuri

4.1.1 Cattolico Giuseppe

Ripercorrendo il lavoro svolto, credo che il nostro approccio di partire dallo sviluppo del Model per poi passare al Controller e infine alla View ci abbia salvato da parecchi mal di testa, soprattutto grazie alla scrittura dei test. Aver testato a fondo le classi del modello fin da subito mi ha infatti permesso di scovare e sistemare un bel po' di bug in anticipo, e quando mi sono trovato a dover collegare il resto dell'interfaccia il lavoro è stato molto più scorrevole. Mettendo le mani in pasta mi sono anche reso conto di quanto i design pattern siano davvero utili nella pratica e non solo concetti teorici; ad esempio, ho sfruttato tantissimo lo Strategy perché mi permetteva di non "ingessare" il codice. Certo, non è stato tutto perfetto: andando avanti con l'implementazione mi sono accorto che in alcune parti, come in certe Factory, avevo pensato troppo poco a come poter estendere il codice in futuro, finendo per violare il principio OCP. Purtroppo rimettere mano alla progettazione da zero in quelle fasi avrebbe richiesto un tempo che non avevamo, ma aver riconosciuto il limite è stata comunque un'ottima lezione. All'interno del gruppo ho cercato di ricoprire un ruolo trasversale, concentrandomi maggiormente sui sincronismi del Controller ma lavorando in modo attivo anche su alcuni aspetti grafici della View. Nel complesso sono molto soddisfatto di come è andata: penso di aver fatto un buon lavoro, di essermi coordinato bene con i miei compagni di progetto e, soprattutto, di aver imparato ad analizzare in modo molto più critico e maturo il software che scrivo.

4.1.2 Muller Arthur

4.1.3 Pulzoni Alessio

Sono soddisfatto del lavoro sul sottosistema dei missili, che ho seguito per intero: dal modello fino a come vengono disegnati a schermo, passando per il caricamento dei loro dati e per la gestione della difficoltà. Lavorando a questa parte ho capito quanto sia utile organizzare bene il codice. Ho fatto in modo che tutti i missili condividessero la stessa "ossatura" comune, lasciando a ogni tipo solo il proprio modo di muoversi: così non ho dovuto riscrivere le stesse cose più volte. E ho strutturato i tipi di missile in modo che aggiungerne uno nuovo costi pochissimo, senza dover toccare il resto del codice già funzionante. La parte di cui sono più soddisfatto è quella che sceglie quale missile far comparire e regola la difficoltà: l'ho scritta come una classe "isolata", che non dipende da nient'altro, e questo mi ha permesso di provarla e verificarla in modo affidabile e ripetibile. Mi ha fatto capire quanto convenga tenere separata la logica dal resto del programma. Riconosco anche un limite: i valori che regolano la difficoltà li ho sistemati a mano provando e riprovando il gioco; una soluzione più pulita sarebbe stata metterli in un file esterno, modificabile senza toccare il codice. Come miglioramenti futuri vedo uno sprite dedicato per ogni tipo di missile e l'aggiunta di nuovi tipi, operazione resa semplice proprio dal modo in cui ho costruito questa parte. Nel gruppo mi sono trovato bene; la difficoltà maggiore è stata tenere la mia parte allineata a una struttura comune che è cambiata durante lo sviluppo, e questo mi ha richiesto diverse sistemazioni e integrazioni.

4.1.4 Spinaci Gianmarco

Dopo una fase iniziale di analisi del dominio, ho progettato e implementato i tre sottosistemi principali di cui mi sono occupato: la gerarchia dei controller con il sistema degli effetti, il motore di collisione con la propagazione degli eventi interni, e l'assemblaggio dello stato di rendering. Nel complesso sono soddisfatto delle scelte architetturali adottate. L'uso del pattern Template Method su EntityControllerImpl si è rivelato particolarmente utile: aggiungere CollectibleControllerImpl ha richiesto solo di sovrascrivere updateEntities, senza toccare la logica di registrazione nel motore di collisione. Allo stesso modo, modellare InternalEventListener come interfaccia funzionale ha reso il CollisionEngine completamente indipendente da chi gestisce gli eventi, una scelta che ha semplificato notevolmente il collegamento tra i sottosistemi durante l'integrazione. Mi sono reso conto di quanto i pattern, se applicati con cognizione di causa, rendano il codice molto più facile da estendere e da spie-

gare agli altri membri del gruppo. Come possibili sviluppi futuri, introdurrei una struttura spaziale nel CollisionEngine per ridurre il costo dei confronti e renderei configurabili da file i parametri di spawn dei collectible, in modo da poter bilanciare il gioco senza ricompilare. Complessivamente ritengo di aver prodotto un buon lavoro, coordinandomi bene con gli altri membri del gruppo nella divisione delle responsabilità tra model, controller e view.

Appendice A

Guida utente

A.1 Avvio e menù principale

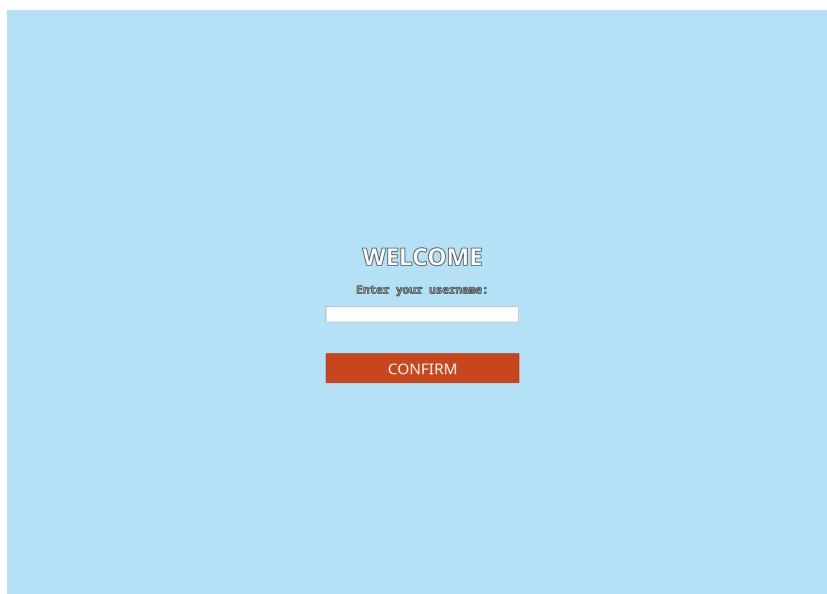


Figura A.1: Schermata iniziale

Al primo avvio l'applicazione mostra la schermata di benvenuto (Figura A.1): è sufficiente digitare un nome utente e premere CONFIRM. Il nome viene salvato e non sarà più richiesto agli avvii successivi, che porteranno direttamente al menù.



Figura A.2: Schermata principale

Il menù principale (Figura A.2) mostra in alto il nome utente, le monete accumulate e il velivolo equipaggiato, e presenta quattro pulsanti

- START — avvia una nuova partita;
- SHOP — apre il negozio dei velivoli;
- LEADERBOARD — mostra la classifica dei punteggi;
- EXIT — chiude l'applicazione.

A.2 Comandi

Durante la partita il velivolo avanza automaticamente; il giocatore ne controlla soltanto la direzione, virando a destra o a sinistra premendo rispettivamente il tasto D e A.

A.3 Svolgimento della partita

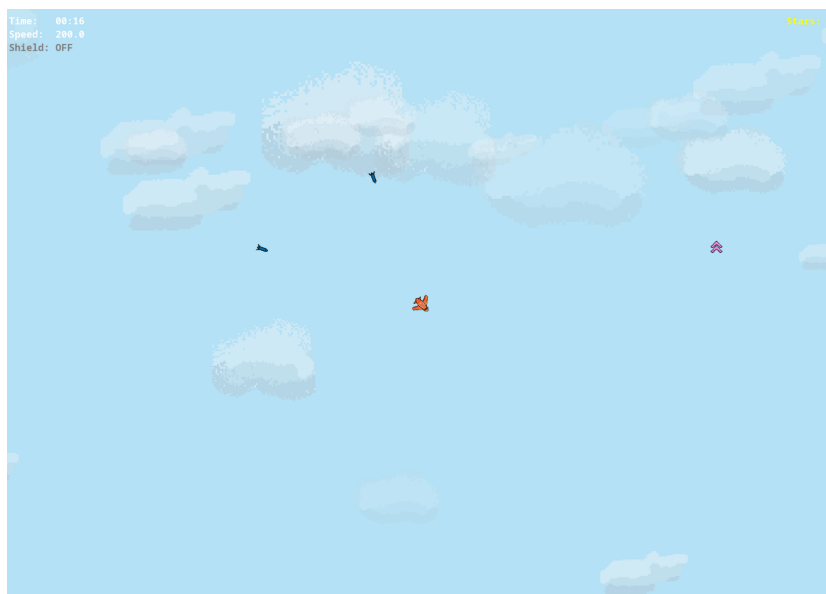


Figura A.3: Schermata di gioco

Avviata la partita (Figura A.3), dai bordi dell'area iniziano a comparire i missili, che inseguono il velivolo aggiornando di continuo la propria rotta. Il giocatore deve schivarli e, manovrando con attenzione, può indurre due missili a scontrarsi tra loro: in tal caso entrambi vengono distrutti. La difficoltà cresce nel tempo, sia nel numero di missili presenti sia nella loro pericolosità; esistono inoltre tipologie di missili diverse, ciascuna con velocità e capacità di manovra proprie. Durante il volo compaiono le entità collezionabili, che si raccolgono passandovi sopra.

- Stella — incrementa il punteggio;
- Boost di Velocità — aumenta temporaneamente la velocità del velivolo;
- Scudo — attiva una protezione a tempo: finché è attiva, il contatto con un missile non causa la sconfitta, indipendentemente dal numero di impatti.

L'HUD in sovrimpressione riporta in tempo reale il punteggio, il tempo di sopravvivenza e lo stato dei potenziamenti attivi. In qualsiasi momento è possibile mettere in pausa con ESC.

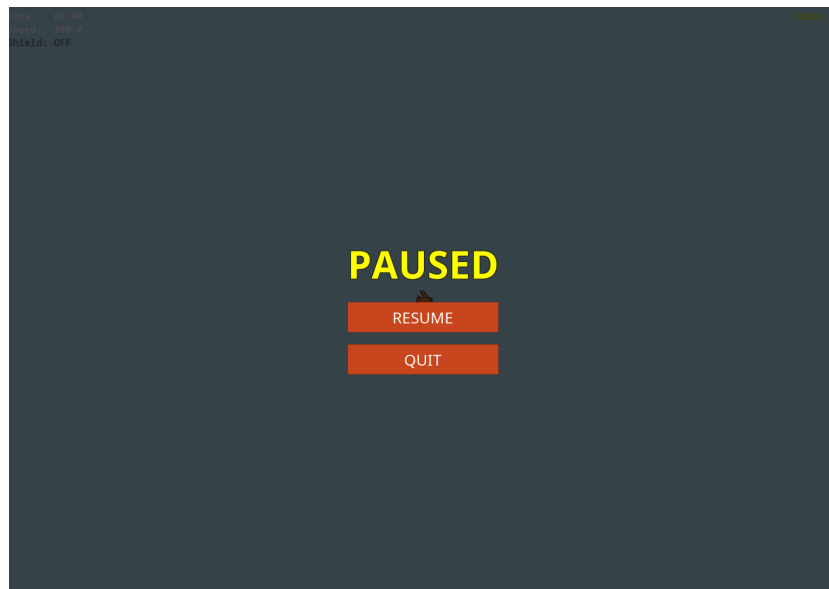


Figura A.4: Schermata di pausa durante la partita

A.4 Negozio

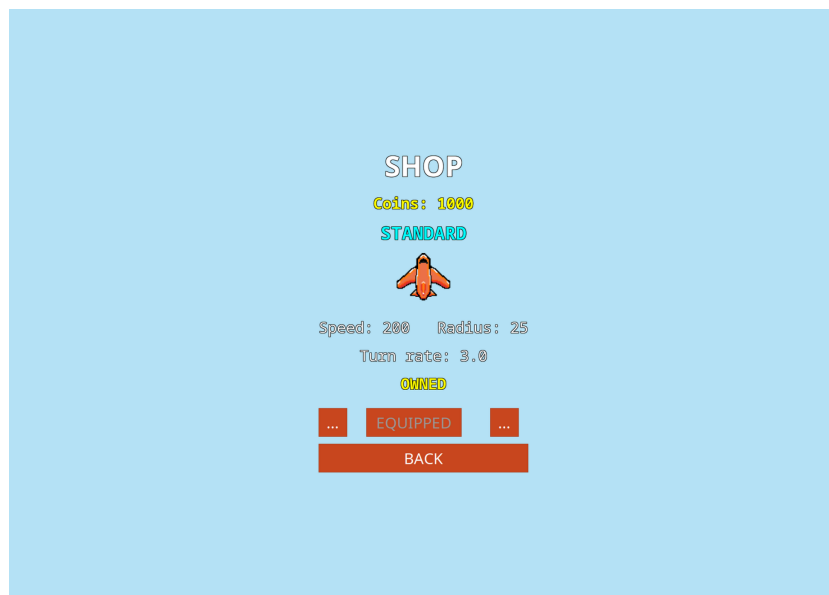


Figura A.5: Schermata di shop

Dal menù, il pulsante SHOP apre il negozio (Figura A.5), dove è possibile spendere le monete accumulate per acquistare velivoli potenziati, dalle caratteristiche superiori a quello base (ad esempio maggiore velocità o virata più stretta). Un velivolo acquistato entra nella dotazione del giocatore e può essere equipaggiato, così da essere usato nella partita successiva.

A.5 Classifica e fine della partita

La classifica (Figura A.6), accessibile dal menù tramite LEADERBOARD, conserva i migliori punteggi conseguiti nelle partite precedenti, salvati su file locale. La partita termina quando un missile colpisce il velivolo in assenza di uno scudo attivo. La schermata di Game Over (Figura A.7) mostra il punteggio finale e la classifica aggiornata con il nuovo risultato, offrendo due possibilità: PLAY AGAIN, per ricominciare immediatamente, oppure MENU, per tornare al menù principale.

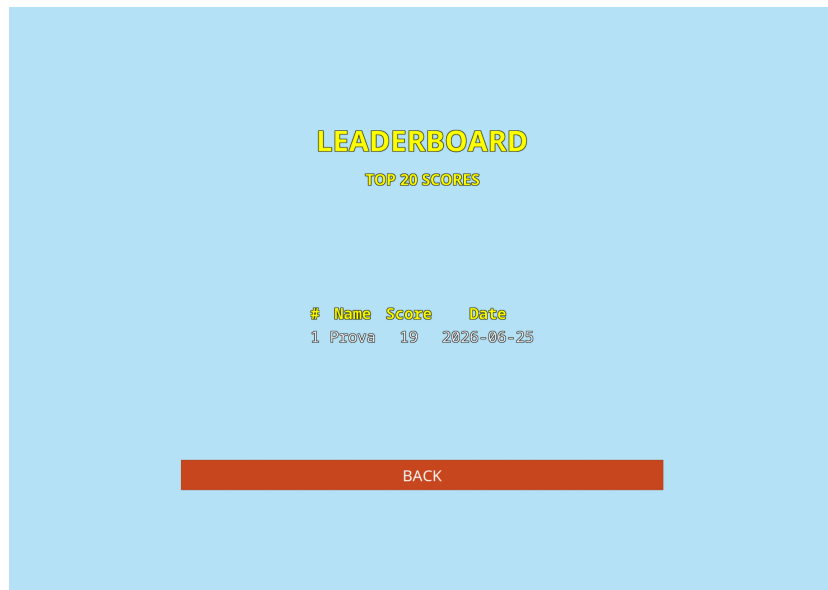


Figura A.6: Schermata di classifica punteggi migliori

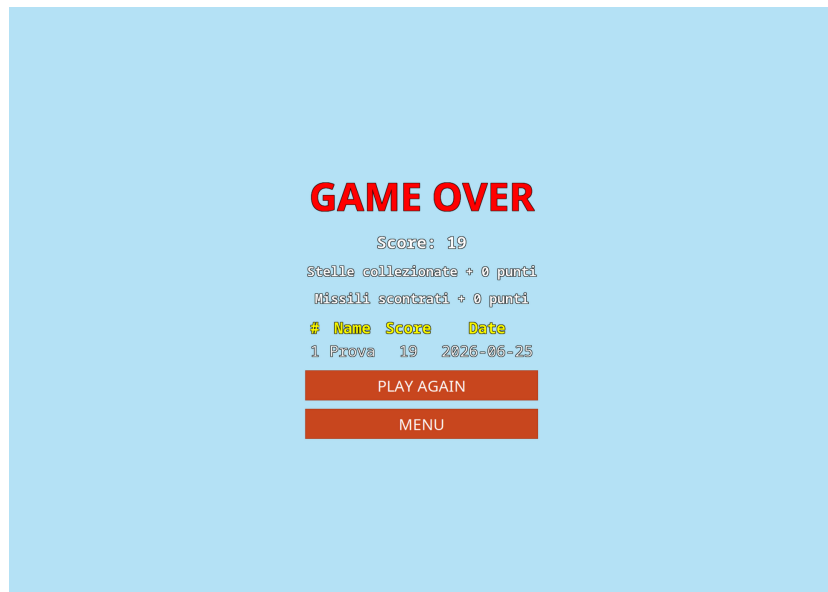


Figura A.7: Schermata di gameover con recap punteggio

Appendice B

Esercitazioni di laboratorio

B.1 `giuseppe.cattolico@studio.unibo.it`

B.2 `arthuristvan.muller@studio.unibo.it`

B.3 `alessio.pulzoni@studio.unibo.it`

B.4 `gianmarco.spinaci3@studio.unibo.it`

Bibliografia

- [1] Kenney. *Kenney Tools*. Disponibile online: <https://kenney.nl/tools>
- [2] Java Design Patterns. *Game Loop Pattern*. Disponibile online: <https://java-design-patterns.com/patterns/game-loop/>
- [3] Conventional Commits. *Conventional Commits v1.0.0: A specification for adding human and machine readable meaning to commit messages*. Disponibile online: <https://www.conventionalcommits.org/en/v1.0.0/>
- [4] Conventional Branch. *Conventional Branch*. Disponibile online: <https://conventionalbranch.org/>
- [5] Google. *Gson: A Java serialization/deserialization library to convert Java Objects into JSON and back*. Disponibile online: <https://github.com/google/gson>